DATA SCIENCE & ANALYTICS

MODULE 05 SESSION 19

DAX measures, filter context and CALCULATE

One measure, many answers: why the number changes.

Tarek Atwan · Lead Instructor

Mon 9 Nov 2026

BY THE END OF TODAY, YOU CAN

Class objectives: measures and their filter context

01 Write explicit measures

02 Divide safely with DIVIDE

03 Explain filter context: one measure, many numbers

04 Add or replace a filter with CALCULATE

05 Compute a percent of total, checked against SQL

EVERYONE DO

Step 1.1

Warm-up quiz on D18’s star model

Five questions, live in Markdown Mash: grain, relationships, the date table, the dashed line, and one on COUNT(DISTINCT).

PART 1

Today’s model: D18’s star, rebuilt from files

Sunday’s tables as four files: you load them and draw the relationships again.



THE MODEL TODAY · PART 1
the star from four files

lime: loaded from the start files
dashed: inactive

INSTRUCTOR DO

Step 1.2

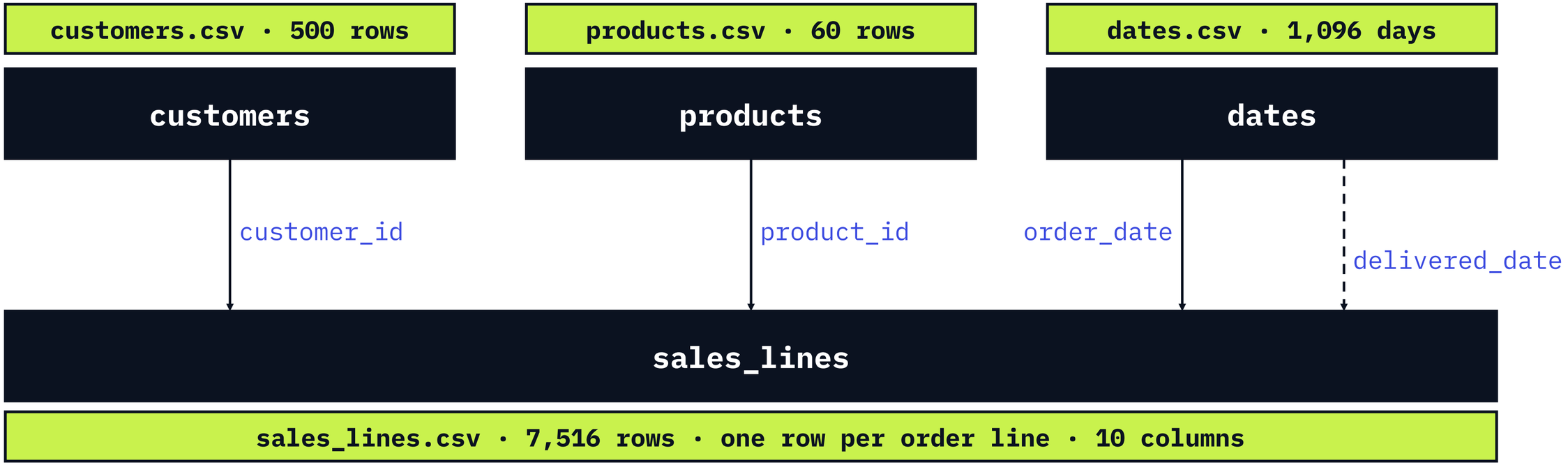
Where today starts: four files instead of Sunday’s file

The same star as Sunday: sales_lines in the middle, customers, products and dates around it.

labs/D19/01-Evr-Load-And-Relate-Start-Files/README.md

TODAY’S MODEL · labs/D19/Resources/start · FOUR CSV FILES, D18’S FINISHED TABLES

Today’s model is D18’s star, loaded from four files



lime: the file you load · **solid line**: active · **dashed**: inactive (Sunday’s delivered date)

The four files are D18’s tables, already cleaned and merged. You load them and draw the four relationships again.

EVERYONE DO

Step 1.3

Activity 01: load the four start files and relate the tables

Load four files, draw four relationships, mark the dates table, and one card: Sum of line_value.

labs/D19/01-Evr-Load-And-Relate-Start-Files/README.md

PART 2

Measures, and how DAX writes them

A calculation you name once: Total Revenue, Orders, Customers and Average Order Value.



THE MODEL TODAY · PART 2
measures

lime: the measures you write on sales_lines

INSTRUCTOR DO

Step 2.1

Measures: what they are and how to write one

Implicit and explicit measures, columns and measures, then Total Revenue written one part at a time.

labs/D19/02-Evr-Base-Measures/README.md

Until today, Power BI summed a column for each visual

CARD

Σ Sum of line_value

filter: status is completed

316,023.50

BAR CHART · BY CATEGORY

Σ Sum of line_value

filter: status is completed

Outerwear	76,637.00
Shoes	62,373.50
Bottoms	50,535.00

… three more categories

CARD

Σ Sum of line_value

no filter: forgotten

340,370.00

Each visual summed line_value on its own and needed its own filter. Forget one, and the number changes.

DEFINITION · MEASURE

Measure: a named calculation run again for each cell

A measure is a named calculation, stored in the model, that Power BI runs again for every card, cell or bar that shows it.

ONE FORMULA, STORED ONCE

```
Total Revenue =
  SUM(sales_lines[line_value])
```

a card · Total Revenue

340,370.00

status	Total Revenue
completed	**316,023.50**
cancelled	**21,998.50**
pending	**2,348.00**
Total	**340,370.00**

The formula is stored once. Each card and each cell runs it again, on its own rows: Part 3 shows how.

IMPLICIT AND EXPLICIT MEASURES · BOTH READ 340,370.00 TODAY

Implicit and explicit measures: what each gives you

IMPLICIT · YOU DRAG A COLUMN

Σ line_value → Sum of line_value

named and made by Power BI, one visual at a time

EXPLICIT · YOU WRITE A MEASURE

Total Revenue = SUM(line_value)

named by you, stored in the model

		implicit	**explicit**
Used inside another measure	Average Order Value is built from it	×	✓
Changed once for every visual	one formula, one place	×	✓
Count distinct, divide, filter	any DAX, not only Sum or Count	×	✓

From here on, every number on a visual is a measure you wrote.

A COLUMN AND A MEASURE · ORDER 100003

A column and a measure: what each one stores

A COLUMN · line_value · ONE VALUE PER ROW

order_id	product_id	qty	price	line_value
100003	P-014	1	24.00	**24.00**
100003	P-060	1	18.00	**18.00**
100003	P-026	1	5.00	**5.00**

stored when the data loads: 7,516 values. Built in Power Query on Sunday; a DAX calculated column stores values the same way

A MEASURE · Total Revenue · NOTHING STORED

order 100003	no filter
47.00	**340,370.00**

computed when a visual asks: one number per cell

A column	to group or filter by a value of each row: category, status, line_value	✓
A measure	to add up, count or divide: Total Revenue, Orders, Average Order Value	✓

No single row holds an average order value, so it has to be a measure.

Total Revenue: the measure’s name comes first

The name is what the Data pane and every visual show. It may contain spaces.

```
Total Revenue =
    SUM(sales_lines[line_value])
```

Total Revenue: SUM adds up one column

SUM adds up every value of one column, on the rows the visual keeps.

```
Total Revenue =
    SUM(sales_lines[line_value])
```

Total Revenue: a column is written as table[column]

The table, then the column in square brackets. Type sales and autocomplete offers both.

```
Total Revenue =
    SUM(sales_lines[line_value])
```

in a card → 340,370.00

WHAT COMES BACK · Total Revenue IN A CARD · THE SAME NUMBER IN BIGQUERY

Total Revenue in a card reads 340,370.00

a card · Total Revenue

340,370.00

```
SELECT SUM(qty * price)
FROM nakheel.order_items;
```

→ 340,370.00, the same number in BigQuery

TOTAL REVENUE BY STATUS · THE THREE PARTS OF 340,370.00



cancelled 21,998.50 · pending 2,348.00

Every line of every status. D16’s 316,023.50 counts completed orders only: activity 02 gets there.

sales_lines HAS ONE ROW PER ORDER LINE (ITS GRAIN, D18) · ORDER 100003

Orders and Customers count distinct IDs, not rows

order_id	customer_id	product_id	line_value
100003	C-1357	P-014	24.00
100003	C-1357	P-060	18.00
100003	C-1357	P-026	5.00

sales_lines · order 100003's three rows

COUNTROWS(sales_lines) → **3 rows**

DISTINCTCOUNT(sales_lines[order_id]) → **1 order**

THE WHOLE TABLE · NO FILTER

COUNTROWS

7,516 rows

Orders · DISTINCTCOUNT(order_id)

3,000 orders

DISTINCTCOUNT(customer_id)

462 customers

One order, three lines. Count the order_id values, not the rows: SQL's COUNT(DISTINCT order_id).

EVERYONE DO

Step 2.2

Activity 02: write the base measures and check them against SQL

Total Revenue, Orders and Customers, then a status slicer that brings all three to the SQL answers.

labs/D19/02-Evr-Base-Measures/README.md

ACTIVITY 02, STEP 3 · THE SAME THREE MEASURES, BEFORE AND AFTER ONE SLICER

With status set to completed, the measures match SQL

NO SLICER

Total Revenue	Orders	Customers
340,370.00	3,000	462

slicer · status = completed

a slicer: a visual you click to filter the whole page · SQL: WHERE status = 'completed'

Total Revenue	Orders	Customers
316,023.50	2,771	452
D16	D07 · completed orders	D09 · buying customers

The same three measures. The slicer chose the rows they add up and count; Part 3 explains how.

INSTRUCTOR DO

Step 2.3

Average order value: why AVERAGE fails, and DIVIDE

Why AVERAGE gives the wrong answer on sales_lines, and how DIVIDE divides safely.

labs/D19/02-Evr-Base-Measures/README.md

A COMMON MISTAKE · AVERAGE ON A TABLE OF ORDER LINES

Average order value is revenue divided by orders

ORDER 100003 · THREE LINES

24.00	18.00	5.00

AVERAGE → 15.67
the average line

47.00 ÷ 1 order → 47.00
the average order

THE WHOLE TABLE · STATUS = COMPLETED

AVERAGE(sales_lines[line_value])	316,023.50 ÷ 6,951 lines	**45.46**
DIVIDE([Total Revenue], [Orders])	316,023.50 ÷ 2,771 orders	**114.05**

The grain of sales_lines decides what AVERAGE averages: lines. Divide the two measures to get the average order.

DIVIDE AND THE / OPERATOR · ROW 2: AN ILLUSTRATIVE EXAMPLE

DIVIDE returns a blank where the slash returns Infinity

The division	with /	with DIVIDE
316,023.50 ÷ 2,771	114.05	114.05
47.00 ÷ 0	**Infinity**	**(blank)**
(blank) ÷ (blank)	(blank)	(blank)

(blank) is DAX's empty value, like SQL's NULL · DIVIDE(top, bottom) · SQL: SAFE_DIVIDE (D04)

The same answer when the bottom number is not zero. Microsoft: use DIVIDE "whenever the denominator is an expression that could return zero or BLANK."

Average Order Value: DIVIDE(top, bottom)

An argument is a value given to a function, inside its brackets: the top number first, then the bottom.

```
Average Order Value =
    DIVIDE([Total Revenue], [Orders])
```

A measure inside a measure goes in square brackets

A measure has no table name in front: [Total Revenue], [Orders]. A column always has one.

```
Average Order Value =
    DIVIDE([Total Revenue], [Orders])
```

no slicer → 113.46 · completed → 114.05

WHAT COMES BACK · Average Order Value · CHECKED AGAINST D04 AND D08

Average Order Value reads 114.05 for completed orders

no slicer · 340,370.00 ÷ 3,000

113.46

status = completed · 316,023.50 ÷ 2,771

114.05

D04 KPI 3 · D08 average_order

```
SELECT AVG(order_total)
FROM nakheel.orders
WHERE status = 'completed';
```

→ 114.05, D04’s third KPI

Completed orders: the same 114.05 as D04’s third KPI and D08’s average_order.

EVERYONE DO

Step 2.4

Activity 02, step 4: Average Order Value with DIVIDE

One measure built from two measures. The completed slicer from step 3 is still on.

labs/D19/02-Evr-Base-Measures/README.md

THE RULE TO KEEP

A measure is written once, and Power BI runs it again for every card, cell and bar.

Total Revenue, Orders, Customers and Average Order Value: four measures, each checked against SQL.

PART 3

Filter context: why one measure shows different numbers

Follow order 100003 through a card, a table and a slicer; then change the filters with CALCULATE.



THE MODEL TODAY · PART 3
filter context

lime: the filters that reach sales_lines from each table

INSTRUCTOR DO

Step 3.1

Filter context: one measure in a card, a table and a slicer

The same measure in three places, where filters come from, how one cell finds its rows, and the total row.

labs/D19/03-Evr-Filter-Context-And-Calculate/README.md

ONE MEASURE, THREE PLACES · STATUS = COMPLETED THROUGHOUT · FOLLOW ORDER 100003

Total Revenue in a card, a table and under a slicer

CARD

316,023.50

100003: all 47.00 inside

TABLE · BY CATEGORY

category	Revenue	100003
Outerwear	76,637.00	**24.00**
Shoes	62,373.50	
Bottoms	50,535.00	
Accessories	43,525.00	
Sportswear	42,365.00	**18.00**
Tops	40,588.00	**5.00**
Total	316,023.50	**47.00**

lime: order 100003’s lines

CARD · CHANNEL SLICER

slicer · channel = online

110,685.50

100003: a store order, 0.00

Same measure, three numbers. Each place keeps a different set of rows, and order 100003 shows which.

DEFINITION · FILTER CONTEXT · WHERE A CELL'S FILTERS COME FROM

Filter context: the filters a measure runs with

Filter context is the set of filters on the model's tables when Power BI calculates one value: one card, one cell, one bar.

channel

☐ store

☑ **online**

1 · a slicer

category	Total Revenue
Outerwear	**25,315.50**
Shoes	25,120.50
Bottoms	16,489.50
Accessories	14,408.00

2 · the visual's rows



3 · a click on a bar

Filters
on this visual
on this page:
status is completed

4 · the Filters pane

The Outerwear cell has three filters: Outerwear from its row, online from the slicer, completed from the Filters pane. It reads 25,315.50.

FILTER CONTEXT AT WORK · THE OUTERWEAR CELL · STATUS = COMPLETED

How the Outerwear cell finds its rows through products

product_id	product_name	category
P-014	**Fleece jacket**	**Outerwear**
P-011	**Rain jacket**	**Outerwear**
P-060	Kids football boots	Sportswear
P-026	Kids T-shirt	Tops

1
*

order_id	product_id	line_value
100003	**P-014**	**24.00**
100003	P-060	18.00
100003	P-026	5.00
100044	**P-011**	**42.00**

1 · the row puts a filter on products[category] = Outerwear

2 · the filter runs along the one-to-many relationship to sales_lines

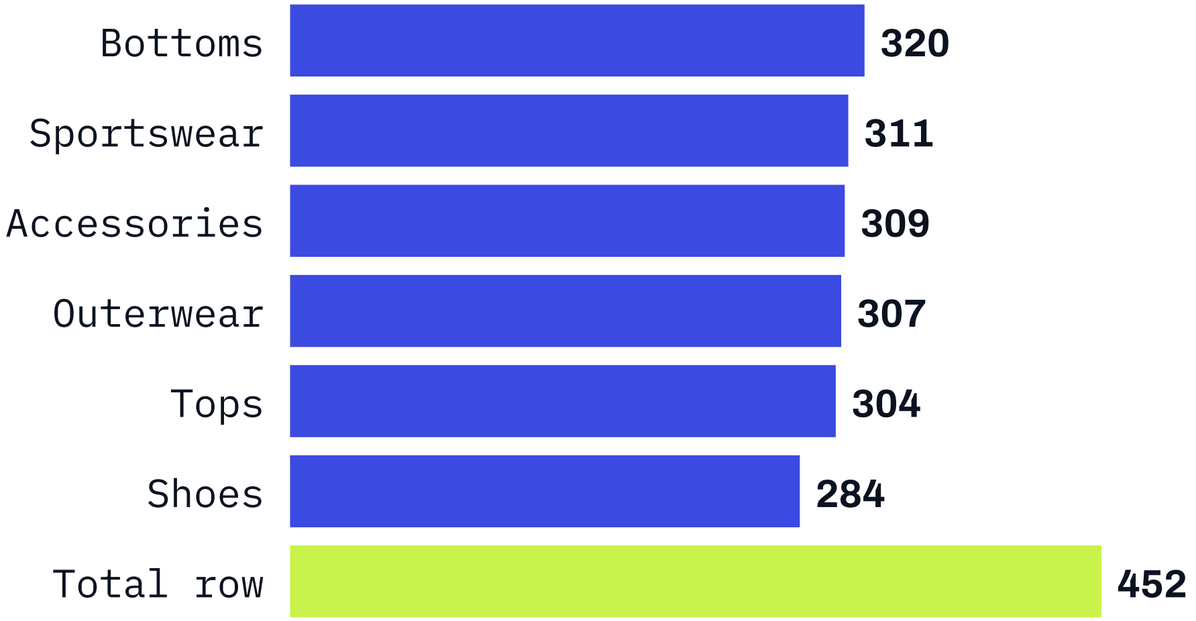
3 · SUM adds the rows left: 1,064 Outerwear lines, the first one order 100003's Fleece jacket

Outerwear → 76,637.00

The measure never sees the other categories: their rows are filtered out before SUM runs. SQL: D06's JOIN, WHERE and GROUP BY, for each cell.

```
Customers = DISTINCTCOUNT(sales_lines[customer_id]) BY CATEGORY · STATUS = COMPLETED
```

The total row is calculated again, not added up



The rows add up to 1,835.

The total row counts each customer once: 452.

C-1357 bought in all six categories: six rows, and once in the total.

Each cell runs the measure on its own rows, and so does the total row. A distinct count’s rows do not add up to its total.

QUICK CHECK

Quick check: which rows does the Shoes cell use under the online slicer?

`slicer · channel = online` `slicer · status = completed`

category	Total Revenue
Outerwear	25,315.50
Shoes	**?**
Bottoms	16,489.50

Name the filters on the Shoes cell. Which rows of sales_lines are left for SUM?

QUICK CHECK · ANSWER

The Shoes cell under the online slicer adds 25,120.50

`category = Shoes`	from the table's row, through products	✓
`channel = online`	from the channel slicer, on sales_lines itself	✓
`status = completed`	from the status slicer, on sales_lines itself	✓

left for SUM: 359 lines of 314 online orders → 25,120.50

channel and status are columns of sales_lines, so their filters need no relationship.

EVERYONE DO

Step 3.2

Activity 03: one measure in a card, a table and under slicers

Revenue and customers by category, then a channel slicer. Steps 1–3; CALCULATE comes next.

labs/D19/03-Evr-Filter-Context-And-Calculate/README.md

INSTRUCTOR DO

Step 3.3

CALCULATE: add a filter or replace one

Write the completed filter into a measure, then see what CALCULATE keeps and what it replaces.

labs/D19/03-Evr-Filter-Context-And-Calculate/README.md

BEFORE AND AFTER · THE COMPLETED FILTER

CALCULATE writes the completed filter into the measure

BEFORE · A FILTER ON EACH VISUAL

Total Revenue	status is completed	316,023.50
Total Revenue	status is completed	Outerwear 76,637.00
Total Revenue	status is completed	Tops 40,588.00

AFTER · ONE MEASURE, Completed Revenue

Completed Revenue	316,023.50
Completed Revenue	Outerwear 76,637.00
Completed Revenue	Tops 40,588.00

The filter is written once, inside Completed Revenue. No visual and no slicer needs it.

DEFINITION · CALCULATE

CALCULATE: a measure run with the filters you write

CALCULATE runs a measure with filters you write. Each filter is added, or replaces the filter already on the same column.

```
CALCULATE(
    [Total Revenue],
    sales_lines[status] = "completed"
)
```

1 · the measure to run (or an expression such as SUM)

2 · a filter: added, or replacing the filter on status

One measure, one or more filters, separated by commas.

Completed Revenue: argument 1 is the measure to run

The first argument is the measure to run: Total Revenue, written once in Part 2.

```
Completed Revenue =
    CALCULATE(
        [Total Revenue],
        sales_lines[status] = "completed"
    )
```

Completed Revenue: argument 2 is a filter on status

The second argument is a filter. DAX text goes in double quotes.

```
Completed Revenue =
    CALCULATE(
        [Total Revenue],
        sales_lines[status] = "completed"
    )
```

with no slicer → 316,023.50

WHAT COMES BACK · TOTAL REVENUE AND COMPLETED REVENUE · NO SLICER

Completed Revenue keeps the category filter and replaces the status filter

BY CATEGORY · THE CATEGORY FILTER IS KEPT

category	Total Revenue	Completed
Outerwear	81,920.50	**76,637.00**
Shoes	67,710.00	**62,373.50**
Bottoms	55,099.50	**50,535.00**
Accessories	46,163.50	**43,525.00**
Sportswear	44,887.00	**42,365.00**
Tops	44,589.50	**40,588.00**

BY STATUS · THE STATUS FILTER IS REPLACED

status	Total Revenue	Completed
completed	316,023.50	**316,023.50**
cancelled	21,998.50	**316,023.50**
pending	2,348.00	**316,023.50**

Each row asked for its own status. CALCULATE asked for completed instead.

Part 4 uses the same replacement on purpose, to put the grand total beside every row.

D18’s Delivered revenue is a CALCULATE measure too

```
Delivered revenue =
CALCULATE(
    SUM(sales_lines[line_value]),
    USERELATIONSHIP(sales_lines[delivered_date],
                    dates[date])
)
```

1 · the measure to run

2 · a filter argument that switches on the dashed delivered-date line



You can now read Sunday’s measure: line_value added up along the delivered-date line, for this measure only.

THE RULE TO KEEP

CALCULATE runs a measure with the filters you write: it adds a filter, or replaces the filter on the same column.

Completed Revenue: 316,023.50 with no slicer, on every status row.

EVERYONE DO

Step 3.4

Activity 03, step 4: Completed Revenue with CALCULATE

Clear the status slicer, write Completed Revenue, and compare it with Total Revenue by status.

labs/D19/03-Evr-Filter-Context-And-Calculate/README.md

REVIEW

Time's up.
Let's review.

Activities 02 and 03: the measures and their SQL

`Solved: labs/D19/03-Evr-Filter-Context-And-Calculate/Solved/solution.md`

Break

Then: percent of total

After the break: each category’s share of completed revenue, and Lab 04.

PART 4

Percent of total with CALCULATE and REMOVEFILTERS

Each category’s share of completed revenue, checked against D10’s window query.



THE MODEL TODAY · PART 4
percent of total

grey ×: the products filter removed for the bottom number

INSTRUCTOR DO

Step 4.1

Percent of total: remove one filter for the bottom number

Why a measure divided by itself gives 100%, then REMOVEFILTERS, the share chart and three bottom numbers.

labs/D19/04-Lab-Percent-Of-Total/README.md

A COMMON MISTAKE · DIVIDE([Completed Revenue], [Completed Revenue])

A measure divided by itself is 100% on every row

category	Completed Revenue	divided by	share
Outerwear	76,637.00	76,637.00	**100%**
Shoes	62,373.50	62,373.50	**100%**
Bottoms	50,535.00	50,535.00	**100%**
Accessories	43,525.00	43,525.00	**100%**
Sportswear	42,365.00	42,365.00	**100%**
Tops	40,588.00	40,588.00	**100%**

The bottom number also sees only the row's category.

For a share, the bottom number needs every category: the same measure with the products filter removed.

THE TOP AND THE BOTTOM OF THE SHARE · THE OUTERWEAR ROW · STATUS = COMPLETED

For the bottom number, remove the products filter

TOP · Completed Revenue · the row's category kept

product_id	category
P-014	Outerwear
P-011	Outerwear
P-060	Sportswear
P-026	Tops

1 *

product_id	line_value
P-014	24.00
P-060	18.00
P-026	5.00
P-011	42.00

76,637.00

BOTTOM · REMOVEFILTERS(products)

product_id	category
P-014	Outerwear
P-011	Outerwear
P-060	Sportswear
P-026	Tops

1 * ✕ filter removed

product_id	line_value
P-014	24.00
P-060	18.00
P-026	5.00
P-011	42.00

316,023.50

76,637.00 ÷ 316,023.50 = 24.3% · D10: revenue / SUM(revenue) OVER ()

The top keeps the row's category. The bottom removes the products filter and sees every category.

DEFINITION · REMOVEFILTERS

REMOVEFILTERS: clear a table’s filters inside CALCULATE

REMOVEFILTERS, used inside CALCULATE, clears the filters on a table or a column for that one calculation. ALL(products) does the same there.

category	Completed Revenue	All Categories Revenue
Outerwear	76,637.00	**316,023.50**
Shoes	62,373.50	**316,023.50**
Bottoms	50,535.00	**316,023.50**
Accessories	43,525.00	**316,023.50**

… the same 316,023.50 on every category row

Every row gets the grand total as its bottom number.

All Categories Revenue: REMOVEFILTERS(products)

Completed Revenue with every filter on the products table removed.

```
All Categories Revenue =
    CALCULATE(
        [Completed Revenue],
        REMOVEFILTERS(products)
    )
```

316,023.50 on every category row

% of Total: DIVIDE the row by all categories

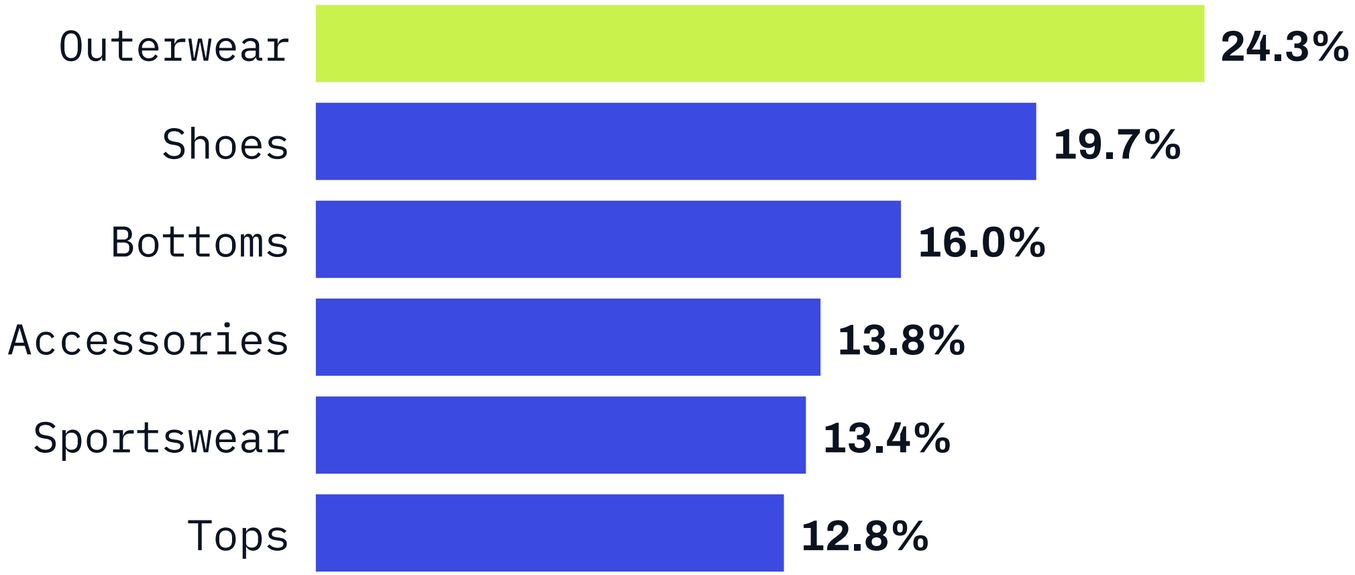
Two measures, so each can be checked. Format % of Total as a percentage.

```
% of Total =
    DIVIDE(
        [Completed Revenue],
        [All Categories Revenue]
    )
```

Outerwear → 24.3%

% OF TOTAL BY CATEGORY · COMPLETED REVENUE · OF 316,023.50

Each category’s share of completed revenue



total 100.0%

```
SQL check (D10):
revenue /
SUM(revenue) OVER ()
```

Outerwear brings almost a quarter of completed revenue. The six shares add up to 100%, as D10’s window query gave.

OUTERWEAR · SLICER channel = online · STATUS = COMPLETED · 25,315.50

The bottom number decides the share: three options

REMOVEFILTERS(products[category])	÷ 110,685.50 = 22.9%	**share of online revenue**
REMOVEFILTERS(products)	÷ 110,685.50 = 22.9% (same)	**share of online revenue**
REMOVEFILTERS()	÷ 316,023.50 = 8.0%	**share of all completed**

REMOVEFILTERS() removes the channel slicer too; Completed Revenue still applies status = completed, so the bottom number is 316,023.50, not 340,370.00.

EVERYONE DO

Step 4.2

Lab 04: percent of total by category, checked against SQL

The 100% dead end, All Categories Revenue, % of Total, then the online slicer. Finish on your own if we run out of time.

labs/D19/04-Lab-Percent-Of-Total/README.md

REVIEW

Time's up.
Let's review.

Lab 04: percent of total by category

`Solved: labs/D19/04-Lab-Percent-Of-Total/Solved/solution.md`

THE RULE TO KEEP

A percent of total divides a cell’s value by the same measure with the products filter removed.

All Categories Revenue reads 316,023.50 on every row; % of Total adds up to 100%.

STUDY LIST · IN CLASS AND IN THE LABS

Today’s DAX functions and where to practise each

In class	What it does	Activity
SUM	adds up one column on the rows a cell keeps	02
DISTINCTCOUNT	counts each order or customer once	02
COUNTROWS	counts rows: lines, not orders	02
AVERAGE	averages rows: the average line	02
DIVIDE	divides; a blank instead of Infinity	02
CALCULATE	runs a measure with filters you write	03
REMOVEFILTERS	clears a table’s filters inside CALCULATE	Lab 04
ALL	the older name; the same inside CALCULATE	Lab 04
USERELATIONSHIP	switches on an inactive relationship	D18

Lab	After class, self-guided, not submitted
04	**Percent of total: finish it if class ran out of time**
05	**Optional, for PL-300: each product’s share of its category**
Reference	**KEEPFILTERS and FILTER: not used today**

CLOSE

What you can do now with DAX measures

- Write a measure, not a column
- Divide safely with DIVIDE
- Name the filters a cell runs with
- Change filters with CALCULATE

NEXT · WED 11 NOV Time intelligence, SUMX, quick measures, report design.